-: Programming in C :-

A Brief History of C:

C is a general-purpose language which has been closely associated with the UNIX operating system for which it was developed - since the system and most of the programs that run it are written in C.

Many of the important ideas of C stem from the language BCPL, developed by Martin Richards. The influence of BCPL on C proceeded indirectly through the language **B**, which was written by Ken Thompson in 1970 at Bell Labs, for the first UNIX system on a DEC PDP-7. **BCPL** and **B** are "type less" languages whereas C provides a variety of data types.

In 1972 Dennis Ritchie at Bell Labs writes C and in 1978 the publication of The C Programming Language by Kernighan & Ritchie caused a revolution in the computing world.

In 1983, the American National Standards Institute (ANSI) established a committee to provide a modern, comprehensive definition of C. The resulting definition, the ANSI standard, or "ANSI C", was completed late 1988.

Structure of a C Program:

Program

```
/* Documentation Section */
// Program: Hello World
// Author: [Your Name]
// Purpose: To print "Hello World" on the screen

#include <stdio.h>   // Preprocessor Directive

// Global Declarations
int main() {          // main() function begins
   // Local Declarations

   // Executable Statements
   printf("Hello, World!\n");

   return 0;          // return 0 means program ended successfully
}
```

Explanation by Sections

1. **Documentation Section**

   /* Program: Hello World */

   - Comments used to describe program name, purpose, author, etc. Comment may be

1.single line → //this is comment

or 2.multiline → /* multiline comment */

2. **Preprocessor Directive**

   #include <stdio.h>

   - Includes **Standard Input Output library**.
   - Needed to use printf() function.

3. **main() Function**

```
int main() {
printf("Hello, World!\n");
return 0;
  }
```

   - Entry point of the program.
   - printf("Hello, World!\n"); prints the message on the screen.
   - return 0; tells the OS that program ran successfully.

Tokens in C:

1. Definition of Token

- A **token** is the smallest unit in a C program that is meaningful to the compiler.
- When we write a program, the compiler breaks it into tokens for processing.

Example:

int sum = a + b;

This statement contains the following **tokens**:

- int → keyword
- sum → identifier
- = → operator
- a, b → identifiers
- + → operator
- ; → special symbol

So, tokens are the **building blocks of a C program**.

2. Types of Tokens in C

C tokens are generally classified into:

1. **Identifiers**
2. **Keywords**
3. **Constants**
4. **Strings**
5. **Operators**
6. **Special Symbols**

Identifiers and Keywords

1. Identifiers

- Identifiers are **names given to various program elements** such as variables, functions, arrays, structures, etc.
- They consist of **letters, digits, and underscores (_)**.
- The **first character must be a letter or underscore**, not a digit.
- C is **case-sensitive**: uppercase and lowercase letters are treated differently.
- Identifiers should be meaningful and not match any reserved keyword.

Example:

int sum, average, A[10];

Here:

- sum → identifier (variable)
- average → identifier (variable)
- A → identifier (array name)

2. Keywords

- Keywords are **reserved words** that have **predefined meanings** in C.
- They cannot be redefined or used as identifiers.
- Example: int, if, while, return, etc.

☞ Example:

```
int main() {
   int number;    // 'int' is a keyword, 'number' is an identifier
   return 0;      // 'return' is a keyword
}
```

3. Reserved Keywords in C

Here is the complete list of 32 keywords in C:

auto	break	case	char	const	continue
default	do	double	else	enum	extern
float	for	goto	if	int	long
register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void
volatile	while				

But, in C99 and later, some new keywords were added:

- _Bool
- _Complex
- _Imaginary
- inline
- restrict

So, in **modern C (C99+)**, the number of keywords is **37**.

Data Types:

Data values passed in a program may be of different types. Each of these data types are represented differently within the computer's memory and have different memory requirements.

The data types supported in C are described below:

Basic Data Types

Data Type	Example	Size (in bytes)	Range	Description
char	char c = 'A';	1	-128 to 127 (signed) 0 to 255 (unsigned)	Stores a single character.
int	int x = 100;	2 or 4 (system-dependent)	-32,768 to 32,767 (2 bytes) -2,147,483,648 to 2,147,483,647 (4 bytes)	Stores integers.
float	float pi = 3.14;	4	~1.2E-38 to 3.4E+38 (6 digits	Stores decimal (real)

Data Type	Example	Size (in bytes)	Range	Description
			precision)	numbers.
double	double d = 10.123456;	8	~2.3E-308 to 1.7E+308 (15 digits precision)	Stores large floating-point numbers.
void	void func();	—	—	Represents no value / empty.

Derived Data Types

Data Type	Example	Size	Description
Array	int arr[5];	Depends on type × elements	Collection of elements of same type.
Pointer	int *ptr;	2 / 4 / 8	Stores memory address of another variable.
Structure	struct student { int id; char name[20]; };	Varies	Groups variables of different types.
Union	union data { int x; float y; };	Size of largest member	Allows storing different data types in the same memory.
Enumeration	enum week {Mon, Tue, Wed};	4 (int)	Defines named integer constants.

Modifier Data Types

Data Type	Example	Size (bytes)	Range
short int	short int a;	2	-32,768 to 32,767
unsigned short	unsigned short a;	2	0 to 65,535
long int	long int a;	4	-2,147,483,648 to 2,147,483,647
unsigned long	unsigned long a;	4	0 to 4,294,967,295
long long int	long long a;	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long	unsigned long long a;	8	0 to 18,446,744,073,709,551,615

Constants:

A constant is an identifier whose value remains unchanged throughout the program. To declare any constant the syntax is:
const datatype varname = value; where const is a keyword that declares the variable to be a fixed value entity.

const datatype variable_name = value;
const → keyword used to declare a constant.
datatype → type of the constant (e.g., int, float, char).
variable_name → name of the constant.
value → the fixed value assigned (cannot be modified later).
Example:

```
#include <stdio.h>
int main() {
   const int MAX = 100;       // Integer constant
   const float PI = 3.1415;   // Floating-point constant
   const char GRADE = 'A';    // Character constant

   printf("Maximum = %d\n", MAX);
   printf("Value of PI = %.4f\n", PI);
   printf("Grade = %c\n", GRADE);

   return 0;
}
```

Types of C operators:

C language offers many types of operators. They are,

1. Arithmetic operators
2. Assignment operators
3. Relational operators
4. Logical operators
5. Bit wise operators
6. Conditional operators (ternary operators)
7. Increment/decrement operators
8. Special operators

Types of Operators	Description
Arithmetic operators	These are used to perform mathematical calculations like addition, subtraction, multiplication, division and modulus
Assignment operators	These are used to assign the values for the variables in C programs.
Relational operators	These operators are used to compare the value of two variables.
Logical operators	These operators are used to perform logical operations on the given two variables.
Bit wise operators	These operators are used to perform bit operations on given two variables.

Conditional (ternary) operators	Conditional operators return one value if condition is true and returns another value is condition is false.
Increment/decrement operators	These operators are used to either increase or decrease the value of the variable by one.
Special operators	&, *, sizeof( ) and ternary operators.

1. Arithmetic Operators

Used for basic mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1

2. Assignment Operators

Used to assign values.

Operator	Meaning	Example	Equivalent
=	Simple assignment	a = 5	a = 5
+=	Add and assign	a += 5	a = a + 5
-=	Subtract and assign	a -= 5	a = a - 5
*=	Multiply and assign	a *= 5	a = a * 5
/=	Divide and assign	a /= 5	a = a / 5
%=	Modulus and assign	a %= 5	a = a % 5

3. Relational Operators

Used for comparison. Always returns **true (1)** or **false (0)**.

Operator	Meaning	Example	Result
==	Equal to	a == b	0 (false)
!=	Not equal to	a != b	1 (true)
>	Greater than	a > b	0
<	Less than	a < b	1
>=	Greater or equal	a >= b	0
<=	Less or equal	a <= b	1

4. Logical Operators

Used for logical decisions.

Operator	Meaning	Example	Result
&&	Logical AND	(a>0 && b>0)	true if both true
||	Logical OR	(a > 0 || b > 0)	true if any one true
!	Logical NOT	!(a>0)	true if false and false if true

5. Bitwise Operators

Works at **bit level** (0s and 1s).

Operator	Meaning	Example	Result
&	Bitwise AND	5 & 3	1 (0101 & 0011 = 0001)
|	Bitwise OR	5 | 3	7 (0101 | 0011 = 0111)
^	Bitwise XOR	5 ^ 3	6 (0101 ^ 0011 = 0110)
~	Bitwise NOT	~5	6 (in 2's complement)→ **~n = -(n+1)**
<<	Left shift	5 << 1	10 (binary shift left)
>>	Right shift	5 >> 1	2 (binary shift right)

6. Conditional (Ternary) Operator

Shorthand for if-else.
Syntax:

```
(condition) ? value_if_true : value_if_false;
```

```
#include <stdio.h>
int main() {
   int a = 10, b = 20, max;

   max = (a > b) ? a : b;   // using ternary operator

   printf("Maximum = %d\n", max);            //Maximum = 20
   return 0;
}
```

7. Increment / Decrement Operators

Used to increase or decrease values.

Operator	Meaning	Example	Result
++a	Pre-increment (increment before use)	a=5; printf("%d", ++a);	6
a++	Post-increment (increment after use)	a=5; printf("%d", a++);	Prints --5 (but a becomes 6)
--a	Pre-decrement	a=5; printf("%d", --a);	4
a--	Post-decrement	a=5; printf("%d", a--);	Prints --5 (but a becomes 4)

8. Special Operators

Some important operators in C:

1. **sizeof operator** → gives size in bytes

```
printf("Size of int = %lu", sizeof(int));
```

2. **Comma operator (,)** → evaluates multiple expressions, last one returns value

```
int a = (1, 2, 3);
printf("%d", a); // 3
```

3. **Pointer operators * and &**

```
int x = 10;
int *p = &x;    // address of x
printf("%d", *p); // value at address (10)
```

```
// Arithmetic

int a = 10, b = 3;

printf("%d", a % b);   // Output: 1


// Relational

if (a > b) printf("a is greater");


// Logical

if (a > 0 && b > 0) printf("Both positive");


// Conditional

int max = (a > b) ? a : b;  // max = 10
```

Basic Input/Output Functions

Function	Description	Example
getchar()	Reads a single character from the standard input (keyboard). Returns the character as an int.	c = getchar();
putchar()	Prints a single character to the standard output (screen).	putchar(c);
puts()	Prints a string followed by a newline character (\n).	puts("Hello");
scanf()	Reads formatted input (char, int, float, string, etc.) from the standard input.	scanf("%d", &num);
printf()	Prints formatted output (char, int, float, string, etc.) to the standard output.	printf("Value = %d", num);

Rules for Using I/O Functions

1. Function names are followed by **parentheses ()** enclosing arguments.
    - Example: scanf("%d", &num);
2. Some functions do not return values (e.g., printf, puts).
3. Functions that return values can be used inside expressions.
    - Example: c = getchar();
4. In scanf(), each variable must be preceded by an **ampersand &**, except for **character arrays (strings)**.
    - Example: scanf("%s", str); (no & for arrays)

Conversion Characters

Conversion characters are used with scanf() and printf() to specify the type of data being read or displayed.

Conversion Character	Meaning / Data Type	Example
%c	Single character	scanf("%c", &ch);
%d	Decimal (signed) integer	scanf("%d", &num);
%e	Floating-point (scientific notation)	printf("%e", 123.45);
%f	Floating-point (decimal notation)	scanf("%f", &x);
%g	Floating-point (shorter of %f / %e)	printf("%g", 123.45);
%h	Short integer	short int x; scanf("%hd", &x);
%i	Integer (decimal, octal, or hexadecimal input)	Input: 10, 012, 0xA
%o	Octal integer	printf("%o", 10); → 12
%x	Hexadecimal integer	printf("%x", 255); → ff
%s	String (sequence of characters, stops at whitespace)	scanf("%s", str);
%u	Unsigned decimal integer	unsigned int n; scanf("%u", &n);

Escape Sequences in C

Escape sequences are **special character combinations** used in C to represent characters that cannot be typed directly or have special meaning (like newline, tab, etc.). They begin with a **backslash** \.

Escape Sequence	Meaning	Example / Output
\n	Newline (moves cursor to next line)	printf("Hello\nWorld"); → Hello World
\t	Horizontal tab (adds space like tab key)	printf("Hello\tWorld"); → Hello World
\b	Backspace (moves cursor one step back)	printf("Helloo\b"); → Hello
\r	Carriage return (moves cursor to beginning of line)	printf("Hello\rWorld"); → World
\\	Backslash character (\)	printf("\\"); → \
\'	Single quote	printf("\'C\'"); → 'C'
\"	Double quote	printf("\"Hello\""); → "Hello"
\?	Question mark (avoids trigraph issues)	printf("\?"); → ?
\0	Null character (string terminator)	"Hello\0World" → prints only "Hello"
\a	Alert (beep sound, may not work on all systems)	printf("\a"); → Beep
\f	Form feed (page break in printers, rarely used)	Moves output to next page (in old printers)
\v	Vertical tab	printf("A\vB"); → A B (below A)

Control Flow Statements in C:

C provides two styles of flow control:

- **Branching** is deciding *what actions to take*.
- **Looping** is deciding *how many times to take a certain action*.

Branching

Branching is so called because the program chooses to follow **one branch or another** based on a condition.

1. if Statement

- This is the simplest form of branching.
- It takes an **expression in parentheses** and a **statement (or block of statements)**.
- If the expression evaluates to **true (non-zero value)**, the statement/block is executed.
- If the expression evaluates to **false (zero)**, the statement/block is skipped.

```
if (expression)
{
   // Block of statements
}
```

Example:

```
int marks = 40;
if (marks >= 35)
{
   printf("Pass");
}
```

2. if with Block of Statements

- When multiple statements need to be executed, enclose them in { }.

```
if (expression)
{
   Block of statements;
}
```

Example:

```
if (marks >= 35)
{
   printf("Pass\n");
   printf("Congratulations!\n");
}
```

3. if-else Statement

- Provides two-way decision making.
- If the condition is **true**, the if block executes; otherwise, the else block executes.

```
if (expression)
{
   Block of statements;
}
else
{
   Block of statements;
}
```

Example:

```
if (marks >= 35)
   printf("Pass");
else
   printf("Fail");
```

4. if-else if Ladder

- Used when there are **multiple conditions** to check.

- Executes the first block whose condition is true.
- If none are true, the else block executes.

```
if (expression)
{
   Block of statements;
}
else if (expression)
{
   Block of statements;
}
else
{
   Block of statements;
}
```

Example:

```
if (marks >= 75)
   printf("Distinction");
else if (marks >= 60)
   printf("First Class");
else if (marks >= 35)
   printf("Pass");
else
   printf("Fail");
```

5. Nested if-else Statement

- An **if or else block** inside another if or else block is called **nested if-else**.
- Useful for checking multiple levels of conditions.

```
if (expression1)
{
   if (expression2)
   {
      Block of statements;   // Executes when both conditions are true
   }
   else
   {
      Block of statements;   // Executes when expression1 is true, expression2 is false
   }
}
else
{
   Block of statements;       // Executes when expression1 is false
}
```

Example:

```
if (marks >= 35)
{
   if (marks >= 75)
      printf("Distinction");
   else
      printf("Pass");
}
else
{
   printf("Fail");
}
```

switch statement:

The switch statement is much like a nested if ..else statement. Its mostly a matter of preference which you use, switch statement can be slightly more efficient and easier to read.

```
switch( expression )
    {
      case constant-expression1:        statements1;
      [case constant-expression2:       statements2;]
      [case constant-expression3:       statements3;]
      [default : statements4;]
    }
```

Example:

```
int day = 3;
switch(day) {
   case 1: printf("Monday"); break;
   case 2: printf("Tuesday"); break;
   case 3: printf("Wednesday"); break;
   default: printf("Invalid day");
}
```

Rules of switch Statement in C:

1. Expression inside switch must be **integer, character, or enum type** (not float/double).
2. case labels must be **constant expressions** (no variables allowed).
3. Each case value must be **unique** (no duplicates).
4. default is **optional** (executes if no case matches).
5. break is **optional** (without it, fall-through happens).
6. Multiple cases can share the **same block of code**.
7. Execution **jumps directly** to the matching case.
8. **Nested switches** are allowed.
9. case and default must be **inside the switch block**.
10. switch can only test **equality**, not relational/logical conditions (<, >, &&, || not allowed).

Using break Keyword in switch-case

- In a **switch statement**, multiple case clauses are used to match values.
- If a condition (case) is met, the program starts executing from that case.
- **By default, execution continues (falls through) into the next case clause** until it encounters a break or the switch statement ends.
- To **exit the switch** immediately after executing a matched case, we use the break keyword.

Looping

Loops provide a way to repeat commands and control how many times they are repeated. C provides a number of looping way.

while loop

The most basic loop in C is the while loop. A while statement is like a repeating if statement. Like an If statement, if the test condition is true: the statements get executed. The difference is that after the statements have been executed, the test condition is checked again. If it is still true the statements get executed again. This cycle repeats until the test condition evaluates to false.

Basic syntax of while loop is as follows:

```
while ( expression )
{
  Single statement
  or
  Block of statements;
}
```

Example:

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

for loop

for loop is similar to while, it's just written differently. for statements are often used to process lists such a range of numbers:

Basic syntax of for loop is as follows:

```
for( expression1; expression2; expression3)
{
  Single statement
  or
  Block of statements;
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
   printf("%d ", i);
}
```

In the above syntax:

- expression1 - Initializes variables.
- expression2 - Conditional expression, as long as this condition is true, loop will keep executing.
- expression3 - expression3 is the modifier which may be simple increment of a variable.

do...while loop

do ... while is just like a while loop except that the test condition is checked at the end of the loop rather than the start. This has the effect that the content of the loop are always executed at least once.

Basic syntax of do...while loop is as follows:

```
do
{
  Single statement
  or
  Block of statements;
}while(expression);
```

Example:

```
int i = 1;
do {
   printf("%d ", i);
   i++;
} while (i <= 5);
```

Difference between while and do…while :

Feature	while Loop	do...while Loop
Condition Checking	Before loop body	After loop body
Execution Guarantee	May execute 0 times	Executes at least once
Syntax	while(condition) { //body }	do { //body } while(condition);
Typical Use	Unknown iterations, may skip execution	Menu-driven programs, user input validation
Example (n=0)	while(n>0) printf("Hi"); → Nothing printed	do { printf("Hi"); } while(n>0); → Prints "Hi" once

break and continue statements

C provides two commands to control how we loop:

- break → exit form loop or switch.
- continue → skip 1 iteration of loop.

You already have seen example of using break statement. Here is an example showing usage of **continue** statement.

```
#include<stdio.h>

main()
{
int i;
int j = 10;

   for( i = 0; i <= j; i ++ )
   {
     if( i == 5 )
     {
       continue;
     }
printf("Hello %d\n", i );
   }
}
```

Difference between break and continue :

Feature	break	continue
Action	Terminates the loop/switch immediately	Skips remaining code in current iteration
Control goes to	First statement after loop/switch	Loop condition check (next iteration)
Works in	Loops + switch	Only in loops
Effect on loop	Ends the loop completely	Keeps loop running, just skips iteration
Example Behavior	Exits loop when condition met	Skips printing certain values but loop continues

---

Functions:

A function is a group of statements that together perform a task. Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task.

A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

The C standard library provides numerous built-in functions that your program can call. For example, **strcat()** to concatenate two strings, **memcpy()** to copy one memory location to another location, and many more functions.

A function can also be referred as a method or a sub-routine or a procedure, etc.

Defining a Function

The general form of a function definition in C programming language is as follows −

```
return_type function_name( parameter_list ) {
        //body of the function
}
```

A function definition in C programming consists of a *function header* and a *function body*. Here are all the parts of a function −

- **Return Type** − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** − This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** − The function body contains a collection of statements that define what the function does.

Function Definition

A function is a block of code that performs a specific task.

Here we create a function called **add()** that takes two integers as parameters and returns their sum.

```
/* function returning the sum of two numbers */
int add(int num1, int num2) {

   /* local variable declaration */
   int result;

   result = num1 + num2;   // addition of two numbers

   return result;          // return the result
}
```

Function Declaration

A **function declaration** tells the compiler about the function name, return type, and parameters.

Syntax:

return_type function_name(parameter_list);

For our add() function, the declaration is:

int add(int num1, int num2);

Parameter names are optional, only types are necessary, so this is also valid:

int add(int, int);

Calling a Function

To use a function, we must **call** it in the main() function (or any other function).
When called:

1. Control passes from main() to the function.
2. The function executes its task.
3. The result is returned back to the caller.

Example: Add Two Numbers Using Function

```
#include <stdio.h>

/* function declaration */
int add(int num1, int num2);

int main() {
   /* local variable definition */
   int a = 10;
   int b = 20;
   int sum;

   /* calling a function to get the sum value */
   sum = add(a, b);

   printf("Sum of %d and %d is: %d\n", a, b, sum);

   return 0;
}

/* function returning the sum of two numbers */
int add(int num1, int num2) {
   /* local variable declaration */
   int result;

   result = num1 + num2;

   return result;
}
```

Output:

Sum of 10 and 20 is: 30

Function Arguments

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.

Formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways in which arguments can be passed to a function −

Call Type & Description
Call By Value This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
Call By Reference This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

By default, C uses **call by value** to pass arguments. In general, it means the code within a function cannot alter the arguments used to call the function.

- Inside a function or a block which is called **local** variables.
- Outside of all functions which is called **global** variables.
- In the definition of function parameters which are called **formal** parameters.

Arrays:

Arrays a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

Declaring Arrays

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows −

Datatype arrayName [arraySize];

Initializing Arrays

You can initialize an array in C either one by one or using a single statement as follows −

double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};

Accessing Array Elements

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example −

double salary = balance[9];

2D array example:

int matrix[2][2] = {{1,2}, {3,4}};

printf("%d", matrix[1][0]); // Output: 3

Multi-dimensional arrays

C supports multidimensional arrays. The simplest form of the multidimensional array is the two-dimensional array.

type name[size1][size2]...[sizeN];
int threedim[5][10][4];

Passing arrays to functions

You can pass to the function a pointer to an array by specifying the array's name without an index.

Way1

void myFunction(int *param) {
  .
  .
}

Way2
void myFunction(intparam[10]) {
  .
  .
}
Way3

```
void myFunction(intparam[]) {
  .
  .
}
```

Return array from a function

C allows a function to return an array.

Pointer to an array

You can generate a pointer to the first element of an array by simply specifying the array name, without any index.

```
double balance[50];

double *p;
double balance[10];

p = balance;
```

Pointers:

A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address. The general form of a pointer variable declaration is −

```
type *var-name;
```

Here, **type** is the pointer's base type; it must be a valid C data type and **var-name** is the name of the pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Take a look at some of the valid pointer declarations −

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

How to Use Pointers?

There are a few important operations, which we will do with the help of pointers very frequently. **(a)** We define a pointer variable, **(b)** assign the address of a variable to a pointer and **(c)** finally access the value at the address available in the pointer variable. This is done by using unary operator ***** that returns the value of the variable located at the address specified by its operand.

```
#include <stdio.h>

int main () {

int var = 20;   /* actual variable declaration */
int  *ip;        /* pointer variable declaration */

ip = &var;  /* store address of var in pointer variable*/

printf("Address of var variable: %x\n", &var  );

   /* address stored in pointer variable */
printf("Address stored in ip variable: %x\n", ip );

   /* access the value using the pointer */
printf("Value of *ip variable: %d\n", *ip );

return 0;
}
```

Output

```
Address of var variable: bffd8b3c
Address stored in ip variable: bffd8b3c
Value of *ip variable: 20
```

1. Pointer Arithmetic in C

Pointer arithmetic means performing arithmetic operations on pointers. Only four arithmetic operators are allowed with pointers:

- ++ (increment)
- -- (decrement)
- + (addition)
- - (subtraction)

When we increment or decrement a pointer, it moves by the size of the data type it points to.

Example:

```
#include <stdio.h>
int main() {
   int arr[5] = {10, 20, 30, 40, 50};
   int *ptr = arr; // Pointer to first element

   printf("Pointer Arithmetic Example:\n");
   printf("Initial pointer points to value: %d\n", *ptr);
```

```
    ptr++; // moves to next integer (4 bytes ahead)
    printf("After ptr++ : %d\n", *ptr);

    ptr = ptr + 2; // jumps two integers ahead
    printf("After ptr + 2 : %d\n", *ptr);

    ptr--; // moves one integer back
    printf("After ptr-- : %d\n", *ptr);

    return 0;
}
```

2. Array of Pointers

An array of pointers is an array in which each element is a pointer. This is useful when dealing with strings or dynamic memory.

Example:

```
#include <stdio.h>
int main() {
    const char *fruits[] = {"Apple", "Banana", "Mango", "Orange"};

    printf("Array of Pointers Example:\n");
    for (int i = 0; i < 4; i++) {
        printf("Fruit %d: %s\n", i+1, fruits[i]);
    }

    return 0;
}
```

Here, fruits is an **array of 4 pointers**, each pointing to a string literal.

3. Pointer to Pointer

C allows us to create a pointer to another pointer (also known as **double pointer**).

Example:

```
#include <stdio.h>
int main() {
    int num = 100;
    int *ptr = &num;   // pointer to integer
    int **pptr = &ptr; // pointer to pointer

    printf("Pointer to Pointer Example:\n");
    printf("Value of num = %d\n", num);
    printf("Value using single pointer = %d\n", *ptr);
    printf("Value using double pointer = %d\n", **pptr);
```

```
    return 0;
}
```

4. Passing Pointers to Functions

Pointers can be passed to functions to allow modification of actual values (pass by reference).

Example:

```
#include <stdio.h>

// Function to swap two numbers using pointers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;

    printf("Before swap: a = %d, b = %d\n", a, b);
    swap(&a, &b); // passing addresses
    printf("After swap:  a = %d, b = %d\n", a, b);

    return 0;
}
```

5. Returning Pointer from Functions

A function can return a pointer. But **be careful**:

- You cannot return a pointer to a **local variable** (because it will be destroyed after function exits).
- You can safely return a pointer to a **static variable** or **dynamically allocated memory**.

Example (Static Variable):

```
#include <stdio.h>

int* getStaticValue() {
    static int num = 42; // static variable (lives for entire program)
    return &num;
}

int main() {
    int *ptr = getStaticValue();
    printf("Returning Pointer from Function Example:\n");
    printf("Value from function: %d\n", *ptr);
```

```
    return 0;
}
```

Strings in C:

What is a String?

- A **string** is a sequence of characters stored in memory and terminated by a **null character '\0'**.
- In C, strings are implemented as **arrays of characters**.
- Example:
- char str[6] = "Hello";

  Here, C stores it as:

  H e l l o \0

Declaration of Strings

We can declare a string in two ways:

1. **With size:**
2. char str[10];

   (Can store up to 9 characters + 1 null character).

3. **Initialization:**
4. char str[6] = "Hello";
5. char str[] = "World"; // size auto-calculated

Input and Output of Strings

Using **printf / scanf**

```
#include <stdio.h>
int main() {
    char name[20];
    printf("Enter your name: ");
    scanf("%s", name);   // reads a single word
    printf("Hello %s", name);
    return 0;
}
```

Note: scanf stops at space, so it cannot take multi-word input.

Using **gets / puts (deprecated but seen in old codes)**

```
gets(name);   // reads complete line
puts(name);   // prints string
```

Using **fgets (recommended for safety)**

```
fgets(name, sizeof(name), stdin);
printf("%s", name);
```

Common String Functions (from <string.h>)

String Functions in C

Function	Description	Example	Output
strlen(str)	Returns length of string (excluding \0)	strlen("Hello")	5
strcpy(dest, src)	Copies src into dest	strcpy(b, "Hi")	b = "Hi"
strncpy(dest, src, n)	Copies first n chars	"Computer" → "Comp" (n=4)	Comp
strcat(s1, s2)	Appends s2 to s1	"Hello " + "World"	Hello World
strncat(s1, s2, n)	Appends first n chars of s2	"Data" + "Stru" (n=4)	DataStru
strcmp(s1, s2)	Compares 2 strings (0 = equal, <0 = s1<s2, >0 = s1>s2)	strcmp("abc","abc")	0
strncmp(s1, s2, n)	Compares first n chars	strncmp("apple","app",3)	0
strchr(str, ch)	Finds **first occurrence** of a character	strchr("Hello",'o')	o World
strrchr(str, ch)	Finds **last occurrence** of a character	strrchr("banana",'a')	a
strstr(str1, str2)	Finds substring inside string	strstr("C Program","gram")	gram
strrev(str)	Reverses string (non-standard in GCC)	strrev("Hello")	olleH

Structures:

Arrays allow to define type of variables that can hold several data items of the same kind. Similarly **structure** is another user defined data type available in C that allows to combine data items of different kinds.

Defining a Structure

To define a structure, you must use the **struct** statement. The struct statement defines a new data type, with more than one member. The format of the struct statement is as follows −

struct[structure tag]{

member definition;
member definition;
...
member definition;
}[one or more structure variables];

Accessing Structure Members

To access any member of a structure, we use the **member access operator (.)**. The member access operator is coded as a period between the structure variable name and the structure member that we wish to access. You would use the keyword **struct** to define variables of structure type.

Example:

```
struct Student {
   int id;
   char name[20];
};
struct Student s1 = {101, "John"};
printf("%d %s", s1.id, s1.name);
```

Structures as Function Arguments

You can pass a structure as a function argument in the same way as you pass any other variable or pointer.

Pointers to Structures

You can define pointers to structures in the same way as you define pointer to any other variable −

struct Books *struct_pointer;

Now, you can store the address of a structure variable in the above defined pointer variable. To find the address of a structure variable, place the '&'; operator before the structure's name as follows −

struct_pointer=&Book1;

To access the members of a structure using a pointer to that structure, you must use the → operator as follows −

struct_pointer->title;

Unions:

A **union** is a special data type available in C that allows to store different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple-purpose.

Defining a Union

To define a union, you must use the **union** statement in the same way as you did while defining a structure. The union statement defines a new data type with more than one member for your program. The format of the union statement is as follows −

union[union tag]{
member definition;
member definition;
...
member definition;
}[one or more union variables];

Accessing Union Members

To access any member of a union, we use the **member access operator (.)**. The member access operator is coded as a period between the union variable name and the union member that we wish to access. You would use the keyword **union** to define variables of union type.

Example:

union Data {

   int i;

   float f;

};

union Data d;

d.i = 10;

printf("%d\n", d.i);

```
d.f = 5.5;

printf("%f\n", d.f);   // overwrites i
```

Dynamic Memory Allocation (DMA):

In C, normally we declare arrays like `int arr[10];` but this size must be **known at compile time**.
Sometimes we don't know the size in advance. For such cases, we use **Dynamic Memory Allocation (DMA)** functions from `<stdlib.h>`.

Functions in DMA:

a) malloc() (Memory Allocation)

Allocates a block of memory.

Does **not initialize memory** (contains garbage values).

Syntax:

```
ptr = (type*) malloc(size_in_bytes);
```

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int n = 5;
    ptr = (int*) malloc(n * sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        return 0;
    }

    printf("Memory allocated using malloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);   // garbage values
    }
    free(ptr);
    return 0;
}
```

b) calloc() (Contiguous Allocation)

Allocates memory **and initializes all elements to 0**.

Syntax:

ptr = (type*) calloc(num_elements, size_of_each);

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int n = 5;

    ptr = (int*) calloc(n, sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        return 0;
    }

    printf("Memory allocated using calloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);   // initialized to 0
    }

    free(ptr);
    return 0;
}
```

c) realloc() (Reallocation)

Used to **resize** previously allocated memory (malloc/calloc).

Syntax:

ptr = (type*) realloc(ptr, new_size_in_bytes);

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int n = 3;
```

```
    ptr = (int*) malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        ptr[i] = i + 1;
    }

    // Resize memory to hold 5 integers
    ptr = (int*) realloc(ptr, 5 * sizeof(int));

    for (int i = 3; i < 5; i++) {
        ptr[i] = i + 1;
    }

    printf("After realloc:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }

    free(ptr);
    return 0;
}
```

d) free() (Deallocation)

Used to **release memory** allocated by malloc, calloc, or realloc.

Prevents memory leaks.

Syntax:

free(ptr);

Function	Memory Init	Parameters	Purpose
malloc	Garbage	Size in bytes	Allocates memory
calloc	Zero	No. of elements, size of each	Allocates & initializes
realloc	Preserves old data	Pointer, new size	Resizes memory
free	N/A	Pointer	Releases memory

Preprocessor Directives:

Preprocessor directives are instructions processed **before compilation**. They start with #.

Common Preprocessor Directives:

#include

Used to include header files.

Example:

```
#include <stdio.h>  // system header file
#include "myheader.h" // user-defined header file
```

#define (Macros)

Used to define constants or macros.

Example:

```
#define PI 3.1415
#define SQUARE(x) (x*x)

int main() {
   printf("PI = %.2f\n", PI);
   printf("Square of 5 = %d\n", SQUARE(5));
   return 0;
}
```

#undef

Undefines a previously defined macro.

```
#define MAX 100
#undef MAX
```

#ifdef, #ifndef, #endif (Conditional Compilation)

Example:

```
#define DEBUG
#ifdef DEBUG
   printf("Debug mode ON\n");
#endif
```

Storage Classes in C:

Storage classes determine **scope, visibility, and lifetime** of variables.

Types of Storage Classes:

auto (default)

Local to block, destroyed when function exits.

Example:

```
void main() {
   auto int x = 10;
   printf("%d", x);
}
```

register

Suggests storing variable in CPU register.

Example:

```
register int i;
for (i = 0; i < 10; i++) {
   printf("%d ", i);
}
```

static

Retains value between function calls.

Example:

```
void demo() {
   static int count = 0;
   count++;
   printf("Count = %d\n", count);
}
```

extern

Declares a global variable defined in another file.

Example (multi-file):

```
// file1.c
int x = 10;

// file2.c
```

```
extern int x;
printf("%d", x);
```

Multi-File C Programs:

Useful for **large projects**.

Consists of **header files (.h)** and **source files (.c)**.

Example:

mathutils.h

```
int add(int, int);
```

mathutils.c

```
int add(int a, int b) {
   return a + b;
}
```

main.c

```
#include <stdio.h>
#include "mathutils.h"

int main() {
   int result = add(5, 10);
   printf("Result = %d\n", result);
   return 0;
}
```

→ Compile:

```
gcc main.c mathutils.c -o program
```

File & Console I/O:

Console I/O

Uses printf() and scanf().

Example:

```
int x;
printf("Enter a number: ");
scanf("%d", &x);
```

printf("You entered %d", x);

File I/O

Files are handled using FILE* pointer.

Basic Functions:

fopen(filename, mode)

fprintf(), fscanf()

fclose()

Example:

```
#include <stdio.h>

int main() {
FILE *fp;
fp = fopen("data.txt", "w");
fprintf(fp, "Hello File Handling!");
fclose(fp);

char str[50];
fp = fopen("data.txt", "r");
fscanf(fp, "%[^\n]", str);
printf("File Content: %s", str);
fclose(fp);
return 0;
}
```

Command Line Arguments:

Arguments passed from the terminal.

main() becomes:

int main(int argc, char *argv[])

argc = number of arguments.

argv[] = array of strings.

Example:

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
printf("Total arguments: %d\n", argc);
for (int i = 0; i < argc; i++) {
printf("Argument %d: %s\n", i, argv[i]);
}
return 0;
}
```

Run:

a.exe hello world

Output:

```
Total arguments: 3
Argument 0: ./a.out
Argument 1: hello
Argument 2: world
```